Prepare answers before the exercise class. In class you will present them.

### 3.1 Measuring clock differences of physical clocks

Measuring the difference between local clocks of two nodes is an important task in clock synchronization algorithms.

a) Explain a simple unidirectional and a simple bidirectional approach for this task. What is the content of the messages and how exactly is the difference calculated?
b) In each case, discuss if/what statements about accuracy (i.e., upper bound on the error) can be made!

**Sample solution.**

***Unidirectional:***

- *A sends its current local time* $T_A$ *to B*
- *upon receiving the message, B records its local time* $T_B$
- *potentially unbounded transmission message delays*
- $\theta = T_A - T_B$ *represents the offset*

*To improve accuracy we can use an estimated message delay* $d$*, and the new offset* $\theta' = T_A - T_B + d$*, but with errors due to variable message transmission delay and inaccuracy of delay estimation.*

***Bidirectional:***

- *Assume that the clock difference between B and A is* $\theta$ *(*$\theta$ *has a positive value if B is ahead of A)*
- *A sends its local time* $T_1$ *to B*
- *B receives the message and stores the local time* $T_2$*,* $T_2 = T_1 + t_{AB} + \theta$*, where* $t_{AB}$ *represents the delay time from A to B*
- *B sends its local time* $T_3$ *to A*
- *A receives the message and stores the local time* $T_4$*,* $T_4 = T_3 + t_{BA} - \theta$*, where* $t_{BA}$ *represents the delay from B to A*

*The total round trip delay can be calculated by adding both equations:*

$$T_2 + T_4 = T_1 + t_{AB} + T_3 + t_{BA} \tag{1}$$

$$\delta = t_{AB} + t_{BA} = T_2 - T_1 + T_4 - T_3 \tag{2}$$

*and by subtracting the equations:*

$$t_{AB} - t_{BA} = T_2 + T_3 - (T_1 + T_4) - 2\theta \tag{3}$$

*Assuming that the communication channel is symmetric, the delays would cancel themselves, which means we can accurately determine the offset.*

$$\theta = \frac{T_2 + T_3}{2} - \frac{T_1 + T_4}{2} \tag{4}$$

*Unfortunately, however, path delay is different between master-to-slave and slave-to-master and this case we have asymmetric delay. The error of the value of the offset ($\theta$) will range between $\pm\delta/2$, i.e., this represents the upper bound of the error. Do note that this upper bound is dependent on the value of the $\delta$, which is computed after receiving all timestamps.*

### 3.2 CNV algorithm

Please download the file `exercise2-cnv.py`. It is a simple (partial) implementation of a simulation of the CNV algorithm. Try to understand the code (do not hesitate to ask if there are any doubts).

The code creates four instances of nodes (A, B, C, D) that use the CNV algorithm to synchronize their local clocks. By small code modifications, you can simulate different situations.

- The unmodified version has four clocks that start in perfect synchrony and have exactly the same speed. It should be no surprise that in this case, all nodes remain in perfect synchrony. You can even remove the line that invokes the CNV synchronization step (`n.sync(nodes)`), and still all will remain perfect.
- Change the node initialization such that one node will have an initial clock error and observe the behaviour of the algorithm.
- Change the node initialization such that one node will have a 5% frequency error and observe the behaviour.
- Change the code such that one node will use the "malicious" EVILCNV implementation (that generates random errors when reading the clock).

Note that in one of the cases, the observed behaviour will not match the expected behaviour (according to the lecture's description of the CNV algorithm).

a) Modify the algorithm such that it correctly implements the CNV algorithm, and repeat above experiment in which you observed a failure.

b) Modify the implementation of EVILCNV such that evil nodes cause maximum damage (maximise the `max diff` value printed during execution)

### 3.3 Lamport's CNV algorithm revisited

"Algorithm CNV. Each process reads the value of every process's clock and sets its own clock to the average of these values—except that if it reads a clock value differing from its own by more than $\delta$, then it replaces that value by its own clock's value when forming the average"[1]

(a) Assume you have 3 processes that execute the CNV algorithm, all of them correct. The parameter $\delta$ is 2 hours. In one synchronization round assume that:

- P1's clock reads 03:00, P2's clock reads 04:00, P3's clock reads 05:00

What will be the clock of P1, P2, and P3, respectively, after the synchronization?

**Sample solution.**

*All obtain the same average value of 5:00*

(b) Assume now that P1 and P3 are the same as above, P2 is malicious. What behaviour of P2 has the worst impact on clock synchronization (i.e., maximizes the difference between P1's and P3's clock) in this example.

**Sample solution.**

*P1's clock has a lower value than P3's, so the worst case behaviour is if P2 sends a small clock value to P1 and a large value to P2. However, if the value is too far away from the recipients local clock, the value will be ignored. So the best case (for the malicious process) is to send 01:00 to P1, and 07:00 to P3, with a difference of just $\delta$ in each case.*

*P1's new clock value will be average of (03:00/01:00/05:00), i.e. 03:00. (unmodified)*

*P3's new clock value will be average of (03:00/07:00/05:00), i.e. 05:00. (unmodified)*

*The clocks will not converge.*

(c) Explain why the CNV algorithm fails to provide convergence in this example!.

**Sample solution.**

*The clock diverged more than before synchronization instead of converging, because the number of participants was too low (less than $n = 3f + 1$ nodes in total with $f = 1$ faulty nodes.*

### 3.4 Synchronisation of physical clocks

a) Which communication protocol (e.g., TCP, UDP, IP) would you use for implementing a clock synchronization algorithm, and why?

**Sample solution.**

*Both NTP and PTP protocol use UDP as transport method. If you want synchronization in a local network, it may be desirable to use Ethernet frames directly (as supported by PTP). Otherwise,*

[1] Leslie Lamport and P. M. Melliar-Smith. Synchronizing Clocks in the Presence of Faults. JACM 32(1), Jan 1985, https://lamport.azurewebsites.net/pubs/clocks.pdf

*to allow packets to traverse routers a higher protocol must be used. You must also take into account that using raw IP or raw Ethernet frames usually requires additional OS privileges.*

*A reliable protocol such as TCP uses message retransmissions, which add an unpredictable delay. This is a very bad idea for a clock synchronization protocol.*

b) Would you implement physical clock synchronization in an application process, in the operating system, or in hardware? What are the respective advantages/disadvantages?

**Sample solution.**

*In terms of accuracy, doing clock synchronization at the lowest level (i.e. hardware) is best. All additional layers (operating systems, application processes) add additional delays due to internal queues, scheduling, etc., and thus decrease accuracy.*

*In terms of implementation complexity, it is usually easier to implement as an application processes than within the operating system, and it is easier to implement in the OS than to do it in hardware.*

c) In the lecture, we discussed that a bidirectional measurement of time differences usually yields a higher accuracy than unidirectional approaches, because you can estimate (and compensate for) the message transmission delay based on measuring the round-trip message delay. Now lets assume that you want to further increase the accuracy by using multiple round-trip measurements.

Assume that the following four measurements $M_1 \ldots M_4$ have been made. Use and explain two criteria to select the best measurement that most likely minimizes the clock offset error. For the selected measurement, calculate the estimated clock offset E.

| base time B [hh:mm:ss] | $t_{C_1}$ [s] | $t_{S_1}$ [s] | $t_{S_2}$ [s] | $t_{C_2}$ [s] |
|---|---|---|---|---|
| M1: 12:05:00.000 | +0.0 | -2.450 | -2.350 | +0.200 |
| M2: 12:10:00.000 | +0.0 | -2.350 | -2.250 | +0.300 |
| M3: 12:15:00.000 | +0.0 | -2.450 | -2.350 | +0.300 |
| M4: 12:20:00.000 | +0.0 | -2.450 | -2.350 | +0.200 |

(Note: all time stamps are shown relative to the "base time", which is the start of the measurement. This means, for example, that for measurement M3, $t_{S_2}$ is 12:15:00.000 – 2.350s = 12:14:57.650.)

$t_{C_1}$: Client sends request

$t_{S_1}$: Server receives request

$t_{S_2}$: Server sends reply

$t_{C_2}$: Client receives reply

**Sample solution.**

*The accuracy of the offset estimation depends on the symmetry of the transmission delays. The more both one-way delays equal each other the better the offset can be estimated. Physical properties like the speed of light or the propagation speed of electrons in a copper wire represent a (usually constant), lower limit for the transmission delay. Besides that the transmission delays are negatively influenced by the quality of the network (queuing delays, collisions, retransmissions, . . . ). These incidents, however, are most of the time asymmetric. Therefore one can assume that the lower the round-trip time is the more symmetric the transmission delays are. This has been experimentally shown by David Mills (see lecture slides 3.42) and has been adapted by the NTP protocol that chooses the measurement with the smallest delay from a series of eight measurements.*

d) Daylight Savings Time (DST) cause "jumps" in local time (by +/- 1 hour once each year). How does the network time protocol account for this?